

FARO — Technical documentation and Canvas implementation

Book 3 of 3 · For whoever installs, develops or deploys FARO

Version: September 2026 · SHIFT 2026 Challenge · Gamification

What this book covers

How FARO is built, module by module, and how it connects to a real Canvas: first to a free Instructure account (what can be done today, in an afternoon), then to an institution's instance through LTI 1.3 (what requires the institution to take part).

Chapters 1 to 5 describe the code. Chapters 6 and 7 are step-by-step procedures. Chapter 8 says how to check everything works, and chapter 9 what is real and what is simulated.

Contents

1. Architecture
2. Code structure and conventions
3. Domain logic (`src/lib/`)
4. The Canvas layer in the extension (`src/data/canvas/`)
5. The backend (`server/`)
6. Canvas implementation, step by step: free account
7. Canvas implementation, step by step: institution with LTI 1.3
8. Verification and tests
9. What is real and what is simulated

Requirements

Tool	Version	For
Node.js	22.18 or later (or 24)	The backend runs TypeScript without compiling (<code>node src/index.ts</code>)
npm	9+	Extension dependencies
Chrome / Edge	current	Loading the extension
A Canvas account	Free-for-Teacher or institutional	Chapters 6 and 7

The commands, in one table

Where	Command	What it does
root	<code>npm install</code>	Installs the extension's dependencies (68 packages)
root	<code>npm run dev</code>	Vite dev server at <code>http://localhost:5173</code>
root	<code>npm run typecheck</code>	Checks types; if it prints nothing after the header, it passed
root	<code>npm run build</code>	Builds the extension into <code>dist/</code>
<code>server/</code>	<code>cp .env.example .env</code>	Creates the backend configuration (then edit it)

Where	Command	What it does
server/	<code>npm run dev</code>	Backend with auto-reload
server/	<code>npm start</code>	Backend
server/	<code>npm test</code>	20 checks against a fake Canvas
server/	<code>npm install</code>	Only if you want <code>npm run typecheck</code> there (installs <code>typescript</code> and <code>@types/node</code> ; the backend itself has no dependencies)

1. Architecture

1.1 The pieces



1.2 The four principles the shape follows

- 1. Canvas is the source of truth; FARO never copies or alters it.** Every FARO open re-reads the course. There is no grade database in FARO, and there will not be: what FARO persists is what FARO creates (purpose, sessions, preferences).
- 2. The credential lives in a single process.** The backend exists for this reason. An extension is public code; a token inside it is a published token. The backend holds the token, decides which paths it accepts and returns JSON. The extension does not know which token exists or what the header is called.
- 3. The seam is a type, not a class.** `CourseSnapshot` (`src/data/client.ts`) is the only way course data enters the rest of the application. Everything above it (`lib/`, `views/`) works on that type and does not know whether it came from fixtures or an institution. That is why switching modes is an environment variable and not a rewrite.
- 4. The logic does not know React exists.** `src/lib/` are pure functions: data in, a decision out. They can be tested without mounting the interface, and can move to the backend unchanged when the friction engine must run on the server.

1.3 The two modes

	mock	http
Chosen with	<code>VITE_CANVAS_MODE=mock</code> (default)	<code>VITE_CANVAS_MODE=http</code>
Who answers the Canvas paths	<code>MockCanvasTransport</code> with <code>fixtures.ts</code>	<code>BackendCanvasTransport</code> → backend → Canvas
Launch (course + person)	Fixed: course 4021, student 77120	<code>GET /api/launch</code> to the backend
Study profile (measured rhythm)	Fixture: 35 min/session, 4 days/week	Absent; Recovery says so
Minutes estimate per activity	Fixture table	20 min default for all
Needs a backend	No	Yes

What does **not** change between modes: `client.ts` (the mapping), `lib/` (all the rules), `views/` (all the screens), `i18n/`. It is the same code against a different transport.

1.4 The flow of one open

1. `main.tsx` mounts `App` inside `StoreProvider`.
2. `store.tsx` reads `chrome.storage.local` (purpose, sessions, preferences, profile).
3. `store.tsx` calls `faroClient.getCourseSnapshot()`.
4. `CanvasFaroClient` resolves the launch (`resolveLaunch`), issues the four requests in parallel and maps the responses to `CourseSnapshot`.
5. `store.tsx` derives everything else with `lib/`: friction, journey, next best action, recovery plan, points, achievements.
6. `App.tsx` shows Purpose if there is no destination; otherwise the active tab.
7. If step 3 fails, `App.tsx` shows the error with a *Retry* button. A course is never invented.

1.5 What the backend does and does not do

Does: hold the token, validate paths, talk to Canvas, paginate, rate-limit, restrict origins, log without personal data, resolve the launch in the pilot.

Does not do yet [pending]: persist FARO state (PostgreSQL), run the friction engine, call the AI model (`POST /api/mentor` exists in the extension as `HttpMentorService`, not in the backend), verify LTI launches (the routes answer 501 with the specification in comments).

The rule for moving something to the backend: when it needs data from more than one device, when it needs a credential, or when it must be auditable by the institution. Until then, it stays in the extension.

2. Code structure and conventions

2.1 The tree

```
faro/
├── public/
│   ├── manifest.json    Manifest V3: side panel, storage; host_permissions localhost only
│   └── background.js    minimal service worker: opens the side panel
├── src/
│   ├── App.tsx          routing between views, bottom bar, loading and error states
│   ├── main.tsx         entry point
│   ├── types/index.ts   ALL domain types, in one file
│   └── data/
│       ├── client.ts    CourseSnapshot + faroClient: the seam
│       ├── canvas/      the Canvas layer (chapter 4)
│       │   ├── endpoints.ts  the 6 paths, the scopes, the audit surface
│       │   ├── raw.ts        types as Canvas returns them (snake_case)
│       │   ├── transport.ts  MockCanvasTransport · BackendCanvasTransport · canvasLog
│       │   ├── client.ts    CanvasFaroClient: the real mapping
│       │   ├── config.ts    mode, backend URL, launch resolution
│       │   ├── status.ts    behaviour flags (activity fallback)
│       │   └── fixtures.ts  the demo course, in Canvas's exact shape
│       └── community.mock.ts
│   ├── lib/             pure logic, no React and no fetch (chapter 3)
│   ├── services/mentor.ts LocalMentorService (today) · HttpMentorService (backend)
│   ├── state/
│   │   ├── store.tsx    the app state and its derivations
│   │   ├── storage.ts   chrome.storage.local with a localStorage fallback
│   │   ├── community.tsx Community state
│   │   └── theme.tsx    light/dark
│   ├── views/           one view per screen
│   ├── components/      reusable pieces (CanvasPanel, Cards, FeedPost...)
│   ├── i18n/            es.ts (source of truth) · en.ts (typed against es) · index.ts
│   ├── styles/          tokens.css · global.css · views.css (plain CSS)
│   └── server/          the backend (chapter 5)
│       ├── src/
│       │   ├── index.ts  start-up
│       │   ├── app.ts    assembles routes
│       │   ├── env.ts    configuration and .env
│       │   ├── allowlist.ts the 6 allowed paths
│       │   ├── canvas.ts  the only function that talks to Canvas
│       │   ├── http.ts    router, CORS, rate limit, logging
│       │   └── routes/    health · launch · canvas · lti
│       ├── test/
│       │   ├── fake-canvas.ts a fake Canvas with real pagination
│       │   └── e2e.ts         20 checks
│       ├── .env.example
│       └── package.json      "dependencies": {}
├── docs/                   the three books
├── .env.example            VITE_CANVAS_MODE, VITE_FARO_API_URL
└── package.json            react, react-dom; vite, typescript
```

2.2 The three seams that matter

lib/ does not import React. If a file in `lib/` needs `useState`, the logic is in the wrong place. The proof: `src/lib/` compiles with `tsc` without `@types/react`.

types/index.ts separates Canvas from FARO. The `Canvas*` types mirror the API. The rest are ours. A field `Canvas` does not have (`estimated_minutes`) carries a comment saying so.

i18n/es.ts is the source of truth. `en.ts` is declared as `const en: Dict` against the Spanish type. A key added in one language and not the other **breaks the build**. That is intentional.

2.3 Conventions

Convention	Rule	Why
Thresholds	Named constants in an exported object (<code>frictionRules</code> , <code>pointsConfig</code> , <code>feasibilityRules</code>)	Tuned in one place and can be shown in documentation
Copy	Always from <code>t.*</code> (the active dictionary), never literals in views	Two languages, and the compiler watches
Components	Small; a <code>.tsx</code> over ~300 lines probably contains <code>lib/</code> logic	Readability
URLs	No hand-written API URL outside <code>data/</code> and <code>server/</code>	One place to change

Convention	Rule	Why
Comments	Explain <i>why</i> , not <i>what</i> . Every non-obvious design decision carries its reason next to the code	Whoever comes next understands without asking
Styles	Plain CSS with tokens in <code>tokens.css</code> ; no frameworks	Small bundle; the extension must feel immediate
TS syntax in <code>server/</code>	Erasable syntax only (<code>erasableSyntaxOnly</code>): no <code>enum</code> , no constructor parameter properties	Node runs the TypeScript directly without compiling
Imports in <code>server/</code>	With an explicit <code>.ts</code> extension	Node requirement for running TS

2.4 Alias

`@/` points to `src/` (configured in `vite.config.ts` and `tsconfig.json`). It is an alias relative to Vite's root, so it works identically on Windows and macOS without path helpers.

2.5 What is NOT there, on purpose

- No Next.js, Electron, Redux or Tailwind. Project instructions: the extension must be lightweight.
- No ORM or database yet.
- No HTTP framework in the backend (Fastify was planned; it was replaced by `node:http` so the token-holding process runs no third-party code; the routes have the shape `(req) => reply`, which ports to Fastify unchanged if the project decides to).
- No test dependency. The backend tests use `node:assert` and run with `node`.

3. Domain logic (`src/lib/`)

Everything FARO *decides* lives here, in pure functions. This chapter documents each module with its inputs, its exact rules and its outputs. The numbers are those in the code; if the code changes, this chapter must change with it.

3.1 `friction.ts` — the friction engine

Input: a `CourseSnapshot` and whether the student is returning after time away. **Output:** a `FrictionSignal` with state, days without activity, pending count, momentum and a supportive headline.

Thresholds (`frictionRules`, configurable):

Constant	Value	Use
<code>frictionDays</code>	3	From here on, FRICTION
<code>disconnectionDays</code>	7	From here on, DISCONNECTION
<code>overwhelmPendingCount</code>	4	Overdue pending items for POSSIBLE_OVERWHELM
<code>momentumDecayWindow</code>	14	Days of inactivity that take momentum from 100 to its floor
<code>momentumFloor</code>	20	Momentum never goes below this

States, in evaluation order:

returning	→ RECOVERY
days ≥ 7	→ DISCONNECTION
pending ≥ 4 and days ≥ 3	→ POSSIBLE_OVERWHELM
days ≥ 3	→ FRICTION
otherwise	→ FLOWING

Days without activity: the most recent timestamp in `snapshot.activity`. If there is none, `disconnectionDays` (7) is assumed: with no evidence of activity, FARO treats the student as disconnected, not as on track.

Pending: assignments with no submission whose due date has passed.

Momentum:

```
decay = min(1, days / 14)
load  = min(1, pending / 8) × 0.2
raw   = (1 - decay) × 100 - load × 100
momentum = max(20, round(raw))
```

One day without activity costs ~7 points. Eight overdue items subtract 20. Nothing takes it to zero.

Product rule: the `headline` comes from the dictionary per state and is always supportive language. There is no "risk" field.

3.2 `journey.ts` — the route and the next best action

`buildJourney(snapshot, friction)` orders assignments by module position and then by id, and assigns each a state:

has a submission	→ completed
no submission and due date passed	→ missed
no submission and module locked	→ locked
the first one that is none of the above	→ current
the rest	→ upcoming

The last assignment of each module is a **checkpoint** (`checkpoint: true`). `progressPercent` = completed ÷ total. `currentIndex` is the first `missed` or `current`. `recalculated` is true when the friction state is not FLOWING.

`nextBestAction(journey, availableMinutes)` — the four-step rule:

1. Open = `missed` + `current` + `upcoming`.
2. Of those, the ones that fit in `availableMinutes`.
3. Of those, the shortest.
4. If none fits, the shortest of all open ones.

The reason shown depends on the case: *reopens the route* if it is `missed`, *fits your time* if it fits, *the shortest* otherwise.

`comebackMilestone(journey)` — the shortest open one, with no time condition. It is the comeback mission: not the most important, but the one least likely to be postponed again.

`recoveryRoute(journey)` — three steps (today / tomorrow / next session) taken from the route.

3.3 `recoveryPlanner.ts` — feasibility and plan

This module answers "*can I still finish?*" with numbers.

`remainingWork(snapshot, journey)` — everything not completed, **including locked modules**. Ignoring them would be the reassuring lie this file exists to avoid. Returns activities, minutes, modules, days until the close and the close date.

`capacityFor(stated, profile)` — what the student sustains per day. What they declared wins; if nothing was declared, the measured rhythm spread across the week: $(\text{minutesPerSession} \times \text{daysPerWeek}) \div 7$, rounded to 5. With neither, 0.

`feasibility(work, capacity)` — the verdict:

```
activities = 0                                → comfortable (finished)
no deadline, or daysLeft ≤ 0, or capacity ≤ 0 → unknown (no guessing)
required = minutes / daysLeft
load      = required / capacity
load ≤ 0.85                                → comfortable
load ≤ 1.40                                → tight
otherwise                                     → not_realistic
```

`feasibilityRules = { comfortableLoad: 0.85, stretchLoad: 1.4 }`. `not_realistic` exists on purpose.

`strategies(work, capacity)` — three paces over the same work, with `PACE = { comfortable: 1, balanced: 1.35, intensive: 2 }` multiplying the required daily minutes. Each strategy computes its days needed and its slack (`buffer = daysLeft - days`). **At most one** carries `suggested: true`: the feasible one closest to the student's capacity. With no known capacity, none is suggested.

`buildPlan(journey, work, dailyMinutes, strategy)` — packs the work into days:

- Priority: `missed` first; then nearest due date; then course order.
- Minimum budget per day: 10 minutes.
- Front-loaded: finishing early leaves the slack *before* the close.
- An activity longer than a whole day is scheduled anyway, taking its own day.
- What does not fit before the close goes to `overflow`. It is shown, never silently dropped.

`todayOneThing(plan)` — the first item of the first day.

3.4 `lifeHappened.ts` — the life state and the intervention

Six states, each with an intervention kind and a minute budget:

State	Intervention	Minutes
<code>less_time</code>	<code>shrink_session</code>	10
<code>overwhelmed</code>	<code>single_action</code>	15
<code>dont_understand</code>	<code>mentor</code>	15
<code>lost_routine</code>	<code>comeback_mission</code>	10
<code>need_break</code>	<code>pause</code>	keeps the current one
<code>ready</code>	<code>continue</code>	20

Nothing here touches dates or academic rules. It only changes the size and shape of the next step.

3.5 `points.ts` and `rhythm.ts` — points

`pointsConfig`:

Constant	Value
POINTS_PER_ACTIVITY	10
POINTS_PER_MODULE	50
STREAK_BONUS	5 (every 3 consecutive days, <code>rhythmMilestoneEvery = 3</code>)
COMEBACK_BONUS	25 (once, if there is at least one session and <code>awayGap ≥ frictionDays</code>)
REWARD_REDEMPTION_RATE	100 points = 1 unit

`learningRhythm(sessions)` counts consecutive days with a session, ending today. Breaking the streak stops adding; it never subtracts.

3.6 `achievements.ts` — the five achievements

Id	Exact condition
<code>the_comeback</code>	<code>awayGap ≥ 3</code> and at least one session exists
<code>back_on_track</code>	at least 2 sessions and <code>momentum ≥ 65</code>
<code>weathered_the_storm</code>	some session with <code>pendingAtStart ≥ 2</code>
<code>smart_session</code>	some session with <code>fitAvailableTime</code>
<code>finisher</code>	<code>progressPercent ≥ 100</code>

While unearned, `hint` is shown, never a reproach.

3.7 `sessions.ts` — FARO's own record

A `StudySession` is what FARO notes down when the student marks a step as done. It stores minutes, how many pending items there were at the start and whether it fitted the declared time. `applySessions` overlays the sessions on the route so momentum, points and achievements respond instantly, without waiting for Canvas to record the submission.

3.8 `community.ts` — feed ranking and SOS routing

`rankFeed(posts, ctx)` orders by: relevance to the student's course and module, whether someone is asking for help and unanswered, and recency. **It ignores reactions on purpose.** `KIND_WEIGHT` puts `help: 5` above `achievement: 2`.

`sosRouting: topic → mentor, time → recovery, people → community`. Three kinds of stuck, three systems.

`communityPointsConfig: POINTS_PER_HELPFUL_MARK: 5, POINTS_PER_COMPLETED_ROOM: 20`. Joining a room gives no points; finishing it does.

3.9 `mentorContext.ts` — the privacy boundary

One function, `buildMentorContext`, that produces the eleven fields of `MentorContext`. If a piece of data is not built here, the mentor does not receive it. It does not read `Profile`. It is the only file Book 1 cites as the mentor's data contract.

4. The Canvas layer in the extension (`src/data/canvas/`)

Seven files. It is the only part of the extension that knows Canvas exists.

4.1 `endpoints.ts` — the audit surface

The paths FARO uses, each built from one function:

```
course(id)           GET /api/v1/courses/:id
modules(id)          GET /api/v1/courses/:id/modules?include[]=items&per_page=100
assignments(id)       GET /api/v1/courses/:id/assignments?include[]=submission&per_page=100
userActivity(id, uid) GET /api/v1/courses/:id/analytics/users/:uid/activity
activeCourses()       GET /api/v1/courses?enrollment_state=active&per_page=100
self()               GET /api/v1/users/self (backend only, when resolving the launch)
```

And two lists for the Canvas administrator: `requiredScopes` in the developer key's `url:GET|/api/v1/...` form, and `ltiScopes` with what FARO requests (`contextmembership.readonly` , `result.readonly`) and what it refuses (`score` , `lineitem`).

`PER_PAGE = 100` is Canvas's maximum: fewer round trips, same data.

4.2 `raw.ts` — Canvas's shape

`snake_case` types that declare **only the fields FARO reads**. Canvas sends many more; they are ignored rather than modelled, so a new Canvas release that adds fields cannot break this layer. When this file and the Canvas documentation disagree, the difference should be visible: that is why it is not translated into the code style.

Important fields:

- `RawModule.items` — present when `include[]=items` was requested. Each item has `content_id` (the id of the assignment it points at) and `completion_requirement.completed` (whether *this* student met it).
- `RawModule.state` — `locked` | `unlocked` | `started` | `completed`. **Only present if the caller is a student in the course.** An instructor does not see it.
- `RawAssignment.submission` — present with `include[]=submission`. A `workflow_state: 'unsubmitted'` is a stub that means *not done*.
- `RawUserActivity.page_views` — an object whose keys are ISO dates rounded to the hour and whose values are counts. FARO keeps the keys and discards the values.

4.3 `transport.ts` — how the request is made

One interface, two implementations:

```
interface CanvasTransport {
  readonly mode: 'mock' | 'http'
  get<T>(path: string): Promise<T>
}
```

MockCanvasTransport — looks the path up in a list of patterns (`fixtureRoutes` in `client.ts`), waits 140 simulated ms and returns the fixture. A path with no pattern produces **404 in the log**, exactly like a wrong path against a real Canvas. A typo in `endpoints.ts` shows; it is not hidden.

BackendCanvasTransport — does `fetch( ${apiUrl}/canvas${path} )` with no authorisation header. The backend paginates; here a single round trip is the whole answer. A non-2xx status throws `CanvasHttpError` with the code and the `error` from the backend's JSON body (`path_not_allowed` , `canvas_error` , `canvas_unreachable` ...).

canvasLog — an in-memory ring buffer of 30 records (`at` , `method` , `path` , `mode` , `status` , `ms` , `items` , `error`). The Profile panel subscribes and draws it. It is never persisted or sent. It is the difference between claiming an integration and showing one.

4.4 `client.ts` — the mapping

`CanvasFaroClient.getCourseSnapshot()`:

1. Resolves the launch (`courseId` , `userId`).

2. Issues the four requests in parallel with `Promise.all`.
3. If the activity one fails with 401/403, not everything fails: it sets `integrationStatus.activityFallback = true` and returns `null` for that part. Any other error does propagate.
4. Walks the modules and their items to build two maps: `moduleOfAssignment` (from `content_id` to module id, only for `Assignment` or `Quiz` items) and `completedByModule`.
5. Filters the assignments down to those some module item points at. **An assignment with no module item is not on the route:** Canvas allows loose assignments, and putting them on the journey would invent structure the teacher did not create.
6. Maps each assignment: `submission` is kept only if `workflow_state !== 'unsubmitted'` and there is a `submitted_at`. That is: **done is what the submission says, not the due date.**
7. Maps activity to timestamped events; or, in the fallback, derives events from the student's own submission dates.
8. `estimated_minutes` comes from the injected estimate function: the fixture table in mock mode, `undefined` (→ 20 min default) in http mode, because fixture ids would match real ids by coincidence.
9. `profile` (measured rhythm) exists only in mock mode. Live, FARO is honest about not knowing the student's rhythm yet.

Three decisions a careless integrator would get backwards, named at the top of the file: module membership comes from the items, not the assignment; "done" comes from the submission, not the date; estimated minutes are FARO's and are stated as such.

4.5 `config.ts` — mode and launch

```
canvasConfig = {
  mode: VITE_CANVAS_MODE === 'http' ? 'http' : 'mock',
  apiUrl: VITE_FARO_API_URL ?? 'http://127.0.0.1:3000',
}
```

There is no token field and there cannot be one.

`resolveLaunch()` returns the `LaunchContext` (course, name, person, `verified`, `source`). In mock it is immediate (`source: 'fixture'`). In http it calls `GET /api/launch` once and caches it; `currentLaunch()` and `onLaunch()` let the Profile panel update when it arrives.

`verified` is `false` in both modes today. Only a signed LTI launch will set it to `true`, and the panel says so.

4.6 `status.ts` — behaviour flags

An observable object with `activityFallback`. It exists so a fallback does not pass silently as if it were the real thing: the Profile panel shows the note *"your account cannot read Canvas analytics, so activity is derived from the dates of your own submissions"*.

4.7 `fixtures.ts` — the demo course

A course in Spanish, *Gestión de Proyectos* (GP-101), 6 modules, 16 activities, in the exact shape Canvas sends. The scenario: an adult five weeks into a course, who stopped logging in `DAYS_INACTIVE = 5` days ago, with `DAYS_UNTIL_COURSE_ENDS = 12` days until the close. It is the exact moment FARO exists for.

With these fixtures the expected numbers are: 16 activities, 7 completed (44%), 9 remaining (4 h 35 min), 12 days, 5 without activity, module 3 started, module 5 locked, state `FRICTION`.

4.8 Switching modes

```
# .env.local in the root (git ignores it)
VITE_CANVAS_MODE=http
VITE_FARO_API_URL=http://127.0.0.1:3000
```

`npm run dev` or `npm run build`. No code file changes.

5. The backend (`server/`)

A Node.js process, with no dependencies, that holds the Canvas token and serves the FARO API. It runs TypeScript directly (`node src/index.ts` , Node 22.18+).

5.1 Configuration (`env.ts`)

Read once at start-up, from `process.env` with a fallback to `server/.env` . The process **refuses to start** without `CANVAS_API_URL` and `CANVAS_ACCESS_TOKEN` , and without `https://` (except localhost).

Variable	Default	What it is
<code>CANVAS_API_URL</code>	— (required)	<code>https://canvas.instructure.com</code> or <code>https://<institution>.instructure.com</code>
<code>CANVAS_ACCESS_TOKEN</code>	— (required)	Token of the student account
<code>FARO_COURSE_ID</code>	empty	Course to open; if absent, the first active enrolment
<code>FARO_HOST</code>	<code>127.0.0.1</code>	Listening interface
<code>FARO_PORT</code>	<code>3000</code>	Port
<code>FARO_ALLOWED_ORIGINS</code>	<code>http://localhost:5173, http://127.0.0.1:5173</code>	Allowed browser origins, comma-separated
<code>FARO_RATE_LIMIT_PER_MINUTE</code>	<code>120</code>	Requests per minute per client
<code>FARO_CANVAS_TIMEOUT_MS</code>	<code>10000</code>	Maximum time for one Canvas call
<code>FARO_LOG_LEVEL</code>	<code>info</code>	<code>info</code> or <code>debug</code>

`describe(env)` is the only representation of the configuration that may be printed or returned: Canvas host, port, origins and `tokenPresent: true` . Never the token.

5.2 Allow list (`allowlist.ts`)

Six patterns, each with its `purpose` written next to it. `allow(pathname, search)` :

1. Decodes the path (`%2e%2e` cannot escape).
2. Rejects `..` and `//` .
3. Looks for a matching pattern; if none, `path_not_allowed` .
4. Rebuilds the query with allowed keys only (`include[]` , `per_page` , `enrollment_state` , `page`) and validated values (`include[]` only `items` or `submission`; `enrollment_state` only `active`).
5. Returns the exact path that will be sent to Canvas.

This file **must match** `src/data/canvas/endpoints.ts` in the extension. If a path is added in one, it is added in the other, and its purpose is documented.

5.3 The Canvas call (`canvas.ts`)

`canvasGet(env, path)` is the only function that builds a request to Canvas:

- `GET` , `Authorization: Bearer` , `Accept: application/json` , `User-Agent: FARO/0.1 (read-only)` .
- `AbortController` with the configured timeout.
- `redirect: 'manual'` : a redirect is not followed (it could carry the token to another host).
- Follows `Link: rel="next"` to the end and concatenates arrays; maximum 50 pages.
- Before each page, verifies the URL is still on `env.canvasApiUrl` .
- Errors as `CanvasError { status, path, kind: 'http' | 'timeout' | 'network' }` . No response body.

5.4 The HTTP layer (`http.ts`)

`HttpApp` on top of `node:http` :

- **Router** by method and regular expression. Unknown path → 404 ; known path with the wrong method → 405 .
- **CORS**: if there is an `Origin` and it is not in the list → 403 `origin_not_allowed` , with no CORS headers. `OPTIONS` preflight → 204 with `GET`, `OPTIONS` .
- **Rate limit**: fixed window per remote address; when exceeded → 429 with `Retry-After: 60` . A sweep every minute discards old windows.
- **Headers** on every response: `Content-Type: application/json; charset=utf-8` , `Cache-Control: no-store` , `X-Content-Type-Options: nosniff` , `Referrer-Policy: no-referrer` , `X-FARO-Request: <id>` .
- **Log**: one JSON line per request with `id` , `method` , `path` (no query), `status` , `ms` . Internal errors are logged with their message and the client receives `500 internal` , never the detail.

Handlers have the shape `(req: Req) => Reply | Promise<Reply>` , with `json(status, body, headers?)` as a helper. That shape ports to Fastify unchanged.

5.5 Routes (`routes/`)

Route	Handler	Response
<code>GET /health</code>	<code>app.ts</code>	<code>{ ok, service, uptimeSeconds, canvas: { host, mode: 'live', readOnly: true }, allowedEndpoints: [{ name, purpose }] }</code>
<code>GET /api/launch</code>	<code>launch.ts</code>	<code>{ courseId, courseName, userId, verified: false, source: 'config' 'enrollment' }</code> . Resolves <code>users/self</code> (keeps only the id) and the course from <code>FARO_COURSE_ID</code> or the first active enrolment. Cached in memory. Invalid token → 502 <code>canvas_token_rejected</code> .
<code>GET /canvas/api/v1/...</code>	<code>canvas.ts</code>	Allow list → <code>canvasGet</code> → full JSON. Headers <code>X-FARO-Canvas-Pages</code> and <code>X-FARO-Canvas-Endpoint</code> . Errors: 403 (not allowed), 401/403/404 (passed through from Canvas), 502 <code>canvas_unreachable</code> , 504 <code>canvas_timeout</code> .
<code>GET /lti/login</code> , <code>POST /lti/launch</code> , <code>GET /.well-known/jwks.json</code>	<code>lti.ts</code>	501 <code>lti_not_configured</code> with the missing step. The full flow specification is in the file's comments.

5.6 Start-up (`index.ts`)

```
[faro-server] listening on http://127.0.0.1:3000 -> Canvas canvas.instructure.com (read-only, token in memory only)
```

Without a valid `.env` :

```
[faro-server] CANVAS_ACCESS_TOKEN is required
[faro-server] Copy server/.env.example to server/.env and fill it in.
```

`SIGINT` / `SIGTERM` close the server cleanly.

5.7 Tests (`test/`)

`fake-canvas.ts` is a fake Canvas that demands the Bearer, paginates with `Link` (2 rows per page to force several pages), and can answer 401 on analytics the way Canvas does for a student without the permission.

`e2e.ts` starts the backend on top of that Canvas and runs 20 checks. It runs with `npm test` and exits non-zero on the first failure. What it checks is, literally, what Book 1 claims.

5.8 Adding a Canvas path (procedure)

1. Justify it: which screen needs it and what minimum data it reads.
2. Add the function in `src/data/canvas/endpoints.ts` and the scope in `requiredScopes` .
3. Add the pattern and its `purpose` in `server/src/allowlist.ts` .
4. Declare only the fields that are read in `src/data/canvas/raw.ts` .
5. Add the fixture in `fixtures.ts` and the route in `fixtureRoutes` (`client.ts`).
6. Add the path to `fake-canvas.ts` and a check to `e2e.ts` .

7. Update Book 1 (chapter 2, data inventory) and this chapter.

If step 1 has no clear answer, the path is not added.

6. Canvas implementation, step by step: free account

This procedure connects FARO to a real Canvas in one afternoon, without asking an institution for anything. It uses **Canvas Free-for-Teacher** (`canvas.instructure.com`), which is a real Canvas instance with the same API.

You need two email addresses: one for the *teacher* account (which creates the course) and another for the *student* account (the one FARO accompanies).

6.1 Create the course (teacher account) — 15 minutes

1. Go to `https://canvas.instructure.com/register` and choose **Free for Teacher**. Complete the sign-up with the first email and confirm.
2. On the dashboard, **+ Course** (or *Start a new course*). Name it, for example *Gestión de Proyectos*, and **Create course**.
3. Create at least **two modules** (*Modules* → **+ Module**). Inside each, **+ → Assignment** → *Create Assignment*, with a name and points. Create 4 or 5 assignments in total. Optional: give one a past due date so FARO has something *overdue* to reopen.
4. **Publish** the course (the *Publish* button on the course home page) and **publish every module and assignment** (the cloud icon in green). An unpublished item does not appear for the student.
5. Note the **course id**: it is in the URL, `https://canvas.instructure.com/courses/1234567` → `1234567`.

6.2 Enrol the student — 5 minutes

Two ways:

- **Invitation:** *People* → **+ People**, type the second email, role *Student*, **Next** → **Add Users**. The student receives an email and accepts.
- **Self-enrolment:** *Settings* → *Course Details* → *More Options* → *Let students self-enroll by sharing with them a secret URL*. Save. A link appears; the student opens it.

With the student account (second email), log in to Canvas, accept the invitation, and **submit at least two assignments** (even with a one-line text). That way FARO has real progress to show.

6.3 Generate the token (student account) — 2 minutes

Logged in as the **student**:

1. **Account** (top left) → **Settings**.
2. Scroll down to **Approved Integrations** → **+ New Access Token**.
3. *Purpose*: `FARO pilot`. *Expires*: set a date (30 days is reasonable). **Generate Token**.
4. **Copy the token now**. Canvas shows it in full only at this moment.

Use the **student's** token, not the teacher's. A teacher's token has no `submission` of its own and no `module state`, and FARO would show a course with no progress. It also limits the blast radius of a leak to what that student sees about themselves (Book 1, chapter 4).

6.4 Configure and start the backend — 3 minutes

```
cd server
cp .env.example .env
```

Edit `server/.env`:

```
CANVAS_API_URL=https://canvas.instructure.com
CANVAS_ACCESS_TOKEN=<paste the student's token here>
FARO_COURSE_ID=1234567
```

Start it:

```
npm start
```

You should see:

```
[faro-server] listening on http://127.0.0.1:3000 -> Canvas canvas.instructure.com (read-only, token in memory only)
```

Check from another terminal:

```
curl http://127.0.0.1:3000/health
curl http://127.0.0.1:3000/api/launch
```

`/api/launch` must return your `courseId`, the course name and the student's numeric `userId`. If it returns `canvas_token_rejected`, the token was mis-copied or has expired.

On Windows use PowerShell: `Copy-Item .env.example .env` instead of `cp`, and `Invoke-WebRequest http://127.0.0.1:3000/health | Select-Object -Expand Content` instead of `curl` (or install curl).

6.5 Build the extension in live mode — 3 minutes

In the project **root** create `.env.local`:

```
VITE_CANVAS_MODE=http
VITE_FARO_API_URL=http://127.0.0.1:3000
```

Option A, dev server (fastest to iterate):

```
npm run dev
```

Open `http://localhost:5173`. That origin is already in `FARO_ALLOWED_ORIGINS` by default.

Option B, extension loaded in Chrome:

```
npm run build
```

Chrome → `chrome://extensions` → *Developer mode* → *Load unpacked* → the `dist/` folder. Note the **extension id** Chrome assigns and add it to `server/.env`:

```
FARO_ALLOWED_ORIGINS=http://localhost:5173,chrome-extension://<id>
```

Restart the backend.

6.6 Verify — 2 minutes

1. Open FARO. Complete the purpose (three questions).
2. **Home** must show your real course name and a progress that matches the student's submissions.
3. **Profile** → **Canvas**: the badge says **Live**, *Course 1234567 · Gestión de Proyectos* appears, and the note says which backend the calls go through. Open the log: four calls with status `200`.
4. In the backend terminal, four log lines with `status: 200` and **no query strings or token**.

If the `/analytics/.../activity` path shows `401` in the log and the panel shows the note about analytics: that is the expected behaviour for a student account in Free-for-Teacher. FARO uses your submission dates as activity. It is not an error.

6.7 Break it on purpose (optional, recommended)

- Revoke the token in Canvas (*Approved Integrations* → *delete*). Restart the backend: `canvas_token_rejected`. FARO shows the error with *Retry*, not an empty course.
- Ask for a forbidden path: `curl http://127.0.0.1:3000/canvas/api/v1/courses/1234567/enrollments` → `403 path_not_allowed`, and **no new line** in the Canvas log: it never reached the network.
- Change `FARO_ALLOWED_ORIGINS` to exclude `localhost:5173` and reload the dev server: `403 origin_not_allowed`.

6.8 Common problems

Symptom	Likely cause	Fix
CANVAS_API_URL must use https://	You used http://	Fix the URL
canvas_token_rejected	Token mis-copied, expired or revoked	Generate a new one
no_active_course	The student did not accept the invitation or the course is unpublished	Accept the invitation / publish the course
Home shows 0% with assignments present	The assignments are in no module, or the token is the teacher's	Add the assignments to modules; use the student token
FARO does not load and the console says CORS	The extension's origin is not in FARO_ALLOWED_ORIGINS	Add chrome-extension://<id> and restart
ERR_UNSUPPORTED_TYPESCRIPT_SYNTAX	Node older than 22.18	Update Node
Modules appear with no state (all unlocked)	Teacher token	Use the student token

7. Canvas implementation, step by step: institution with LTI 1.3

This is the real deployment path. A browser extension cannot be installed institutionally; an LTI tool can. That is why the plan is **LTI 1.3 first, the extension as an optional layer** for those who prefer it.

What follows has two parts: what the institution does (register the tool) and what the FARO team does (implement three routes that answer 501 today). Neither is done yet; both are specified.

7.1 What FARO asks of the institution

A Canvas administrator creates a **Developer Key** → **LTI Key** with these values:

Field	Value
Method	Manual Entry
Title	FARO
Redirect URIs	<code>https://<faro>/lti/launch</code>
Target Link URI	<code>https://<faro>/lti/launch</code>
OpenID Connect Initiation Url	<code>https://<faro>/lti/login</code>
JWK Method	Public JWK URL: <code>https://<faro>/.well-known/jwks.json</code>
LTI Advantage Services	☒ <code>contextmembership.readonly</code> (NRPS) · ☒ <code>result.readonly</code> (AGS) · ☐ <code>score</code> · ☐ <code>lineitem</code>
Placements	Course Navigation
Privacy Level	Anonymous or Public as the institution decides (see 7.4)

And a **Developer Key** → **API Key** with exactly the scopes from `src/data/canvas/endpoints.ts` (`requiredScopes`), with *Enforce Scopes* enabled:

```
url:GET|/api/v1/courses/:id
url:GET|/api/v1/courses/:course_id/modules
url:GET|/api/v1/courses/:course_id/assignments
url:GET|/api/v1/courses/:course_id/analytics/users/:student_id/activity
url:GET|/api/v1/courses
url:GET|/api/v1/users/self
```

The administrator hands FARO: `client_id`, `deployment_id`, the instance URL and Canvas's JWKS URL (`https://<institution>.instructure.com/api/lti/security/jwks`).

7.2 What FARO implements (the three routes)

All with `node:crypto`, no dependencies. The specification is in `server/src/routes/lti.ts`.

GET /lti/login — third-party OIDC initiation. Receives `iss`, `login_hint`, `target_link_uri`, `client_id`, `lti_message_hint`. Generates a random `state` and `nonce`, stores them with a short expiry (memory or database), and redirects (302) to `https://<institution>.instructure.com/api/lti/authorize_redirect` with `scope=openid`, `response_type=id_token`, `response_mode=form_post`, `prompt=none`, `redirect_uri`, `client_id`, `login_hint`, `state`, `nonce`, `lti_message_hint`.

POST /lti/launch — `id_token` verification.

1. Compares `state` with the stored one; consumes it.
2. Decodes the JWT; takes `kid` from the header; obtains the public key from Canvas's JWKS (cached); verifies the RS256 signature with `crypto.verify`.
3. Verifies `iss` (the instance), `aud` (the `client_id`), `exp`, `iat`, and that `nonce` is the issued, unused one.
4. Reads `https://purl.imsglobal.org/spec/lti/claim/context` → `id` (course) and `sub` (person). Optionally the NRPS and AGS endpoints from their respective claims.
5. Creates a FARO session (`HttpOnly`; `Secure`; `SameSite=None` cookie) bound to (course, person, instance).
6. Redirects to the FARO interface.

GET /.well-known/jwks.json — FARO's public key. FARO generates an RSA key pair on deployment (stored as a secret), publishes the public one here, and signs with the private one the `client_assertion` (RFC 7523) it sends to

`https://<institution>.instructure.com/login/oauth2/token` with `grant_type=client_credentials` to obtain LTI service tokens (NRPS/AGS).

7.3 Per-student API token (OAuth2)

For the six REST paths, with the API Key:

1. After the launch, if there is no token for that person, FARO redirects to `https://<institution>.instructure.com/login/oauth2/auth?client_id=...&response_type=code&redirect_uri=https://<faro>/oauth/callback&scope=<the six scopes>&state=...`.
2. Canvas asks the student for consent (once) and returns a `code`.
3. FARO exchanges it at `POST /login/oauth2/token` for an `access_token` (1 hour) and a `refresh_token`.
4. Both are stored **only in the backend**, encrypted at rest, bound to the person.
5. On expiry, FARO renews with the `refresh_token`. On consent withdrawal, `DELETE /login/oauth2/token`.

With this, `CANVAS_ACCESS_TOKEN` disappears from `.env`: the backend goes from one shared token to one token per person.

7.4 What changes in the backend

Today (pilot)	Production
<code>GET /api/launch</code> infers course and person from the <code>.env</code> token	Reads them from the session created by <code>/lti/launch</code> ; <code>verified: true</code>
One token for everything	One OAuth2 token per person, stored encrypted
Authorisation by origin (CORS) and localhost	Authorisation by session (cookie) + CORS
FARO state in the browser	PostgreSQL: purpose, sessions, preferences, tokens
Local mentor	<code>POST /api/mentor</code> in the backend, with the 11-field context and the provider contract

`allowlist.ts`, `canvas.ts` and the `/canvas/*` routes **do not change**: the allow list and pagination are the same with any token.

Privacy Level of the LTI Key. *Anonymous* sends no name or email in the launch; FARO does not need them, so it is the option consistent with Book 1. *Public* would send them; FARO would ignore them, but better not to receive them.

7.5 Suggested order of work

1. Deploy the backend behind TLS (own domain, valid certificate). Without this, Canvas does not accept the tool.
2. Generate FARO's key pair; publish `/.well-known/jwks.json`.
3. Implement `/lti/login` and `/lti/launch` against a test course in the institution's instance (or in Free-for-Teacher, which also admits LTI Keys from the teacher account).
4. Implement the OAuth2 flow from 7.3.
5. Replace the launch with the session in `/api/launch`.
6. Add PostgreSQL and move FARO's state.
7. Register the tool at account level; pilot with one course.

7.6 The extension in this scenario

It keeps existing as an option: whoever prefers the side panel over the Canvas iframe installs it, and the extension calls the same backend with the same session (the cookie travels if `FARO_ALLOWED_ORIGINS` includes the extension id and the cookie is `SameSite=None`). The extension code does not change; only where `VITE_FARO_API_URL` points changes.

8. Verification and tests

8.1 What is verified automatically

`cd server && npm test` starts a fake Canvas with real pagination and the backend on top, and runs these 20 checks:

```
ok health names the Canvas host and never the token
ok launch resolves the token owner and the first active course
ok course proxy returns the Canvas object
ok modules are paginated by Canvas and merged by the backend
ok assignments carry submissions across three pages
ok activity is forwarded for the token owner
ok a path outside the allow list is refused locally (no Canvas call)
ok a write is refused even on an allowed path
ok an unknown query parameter is refused
ok path traversal is refused
ok a browser origin outside the list gets 403 and no data
ok CORS preflight succeeds for an allowed origin
ok responses carry no-store and nosniff
ok Canvas 404 is passed through as 404
ok the token was sent to Canvas on every call, and only there
ok log lines carry no query strings
ok rate limit trips after the configured burst
ok Canvas 401 on analytics is passed through so the client can fall back
ok a wrong token is reported as rejected, not as a generic failure
ok .env parser handles comments, quotes and CRLF
```

20 checks passed

It uses no test framework: `node:assert` and a ten-line `check(name, fn)`. The process exits non-zero on the first failure, so it works as a CI step.

8.2 What was verified of the extension's mapping

In addition, the `src/data/canvas/` layer was compiled and run in Node against the same fake Canvas and the real backend, in the four scenarios that matter:

Scenario	Expected and obtained result
Mock	16 activities, 7 completed, 4 fixture calls, launch <code>fixture</code> , study profile present
Http through the backend	Course GP-101, 3 modules (the third <code>locked</code>), 5 assignments (orphan 999 excluded), 3 with a submission, correct <code>module_id</code> , <code>course_id</code> from the live course, 20 min default, 3 activity events ordered newest first, no study profile, <code>activityFallback = false</code> , 4 calls with 200
Analytics forbidden (401)	<code>activityFallback = true</code> , activity derived from 3 submissions, the analytics path with 401 in the log, no call to <code>/enrollments</code>
Backend down	<code>getCourseSnapshot()</code> rejects; it never returns an empty course

This script is not in the repository because it depends on compiling the extension without Vite; the scenarios are described here so they can be reproduced.

8.3 Manual verification in the interface

With the fixtures (mock mode), the app must show exactly:

Data	Value
Total activities	16
Completed	7
Progress	44%
Remaining	9 activities, 4 h 35 min
Days until the close	12
Days without activity	5
Friction state	FRICTION

Data	Value
Module 3	started
Module 5	locked

And in **Profile** → **Canvas**: *Simulated* badge, four GET paths, two refused scopes, a log with four `200` entries (~140 ms).

8.4 Break it on purpose

Three experiments with the fixtures, each with a result that must change:

1. **DAYS_INACTIVE from 5 to 9** → DISCONNECTION state, Home changes tone, Recovery offers a comeback mission. None of this is written in the views: it comes from the `disconnectionDays: 7` threshold.
2. **DAYS_UNTIL_COURSE_ENDS from 12 to 3** → *not realistic* verdict with all three strategies marked infeasible and work in overflow.
3. **A typo in a path in `endpoints.ts`** → 404 in the panel's log. The mock does not resolve by coincidence.

8.5 Typecheck

- Root: `npm run typecheck`. Requires a prior `npm install` (React, Vite). If it prints nothing after the two header lines, it passed.
- `server/`: `npm install && npm run typecheck`. Installs only `typescript` and `@types/node` as dev dependencies; the running backend still has no dependencies.

`en.ts` typed against `es.ts` makes a missing key in one language a compile error, not a blank text in the demo.

8.6 What is not tested yet

- The React interface against a real backend in a browser (the tests above cover the data layer, not the render). Covered by the procedure in chapter 6.
- The LTI routes (they answer 501 by design).
- Behaviour with large courses (more than 100 modules or assignments, i.e. more than one page of real Canvas). Pagination is tested with the fake Canvas; it is worth repeating against Free-for-Teacher with `per_page=2` forced in a test.

9. What is real and what is simulated

This list exists so that nobody — not the jury, not an institution, not the team six months from now — believes the prototype does something it does not.

Real and working

Component	State
Backend with token, allow list, pagination, CORS, rate limit, logs without sensitive data	Implemented; 20 automated checks
Extension in <code>http</code> mode against the backend, with no credentials and no permission for <code>instructure.com</code>	Implemented; data layer verified end-to-end
Canvas-to-FARO mapping (modules from items, done from submission, orphans excluded)	Implemented and verified with paginated responses
Activity fallback from submissions when Canvas denies analytics, with an on-screen note	Implemented and verified
Error state with retry when the backend does not answer	Implemented
Friction engine, journey, next best action, planner, points, achievements, feed ranking, SOS routing, mentor contract	Implemented as pure functions; configurable thresholds
es/en i18n verified by the compiler	Implemented
Live request log in Profile	Implemented

Ready to run, pending credentials

Component	What is missing
Connection to Canvas Free-for-Teacher	A student token and the course id (chapter 6, 30 minutes)

Simulated, and labelled as such

Component	What is missing
Course data in mock mode	Comes from <code>fixtures.ts</code> . The badge says <i>Simulated</i> .
Launch	<code>verified: false</code> in both modes. Live, course and person are inferred from the token; the panel says so.
Institutional account (SSO)	Does not exist. The screen labels it as simulated.
Community	Classmates, posts, rooms and mission are sample data. Rooms run on an accelerated clock for the demo.
Study profile (measured rhythm)	Mock only (35 min/session, 4 days/week). Live, absent and declared.
<code>estimated_minutes</code>	Does not exist in Canvas. Own table in mock; 20 min default live.
Mentor	Local and deterministic, no model. <code>HttpMentorService</code> exists in the extension; <code>POST /api/mentor</code> does not exist in the backend.

Designed, with the specification in the code, not implemented

Component	Where the specification is
LTI 1.3: <code>/lti/login</code> , <code>/lti/launch</code> , <code>/.well-known/jwks.json</code>	<code>server/src/routes/lti.ts</code> ; chapter 7
OAuth2 per student	Chapter 7.3; scopes in <code>endpoints.ts</code>
PostgreSQL and cross-device sync	Project instructions; chapter 7.4
Remote mentor behind the backend	<code>services/mentor.ts</code> (<code>HttpMentorService</code>); Book 1, chapter 5
Warning on the destination sentence	Book 1, chapter 5.2

The honesty rule

No validation figure in this project is invented. Where there is no measurement, the documents say *to be measured*. The friction thresholds are prototype rules, configurable, and are not presented as scientifically validated. What is claimed as implemented has code and, when verifiable, a test that can be run.